

Интерактивные задачи

В обычных задачах можно считать (например при тестировании), что Ваша программа считывает данные из одного файла и записывает в другой файл. Но в интерактивных, это не так, и Ваша программа взаимодействует с программой жюри, и получает следующие данные только когда обработала предыдущие (согласно протоколу взаимодействия). Можно такие задачи тестировать вручную, но на больших тестах не хочется тестировать вручную, так что разберёмся как автоматизировать.

Будем считать, что у нас есть программа `jury` (программа жюри) и `solve` (решение участника). Тут уже не понятно как бы можно перенаправить ввод-вывод из одной программы в другую с помощью стандартных консольных `>` `<`. Вместо этого мы сделаем две «*pipe*» (трубы), каждая из которых перенаправят ввод одной программы на вывод другой. Для этого нужна страшная программа, которая даже компилируется и как C++, и как C:

```
1  #include <fcntl.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <sys/wait.h>
5  #include <unistd.h>
6
7  int fd1[2], fd2[2];
8
9  void closing() {
10     close(fd1[0]);
11     close(fd1[1]);
12     close(fd2[0]);
13     close(fd2[1]);
14 }
15
16 int main() {
17     pipe(fd2);
18     pipe(fd1);
19     if (!fork()) {
20         dup2(fd1[0], 0);
21         dup2(fd2[1], 1);
22         closing();
23         execlp("./jury", "jury", NULL);
24         exit(-1);
25     }
26     if (!fork()) {
27         dup2(fd2[0], 0);
28         dup2(fd1[1], 1);
29         closing();
30         execlp("./solve", "solve", NULL);
31         exit(-1);
32     }
33     closing();
34     wait(NULL);
35     wait(NULL);
36     return 0;
37 }
```

Тут происходят разные необычные системные вызовы `pipe`, `fork`, `dup2`, `execlp`, `wait` – желающие могут сами разобраться что они делают. Правда, как известно, системные вызовы платформозависимы, поэтому код скомпилируется только под Linux, не под Windows.

Допустим, что у нас задача – угадать число в диапазоне `[1, 1000]`, тогда программа `jury` может выглядеть например так:

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int secret = 123;
5      while (true) {
6          int n;
7          cin >> n;
8          cerr << "Jury: got=" << n << endl;
9          char ans;
10         if (n == secret) ans = '=';
11         else if (n > secret) ans = '<';
12         else ans = '>';
13         cerr << "Jury: ans=" << ans << endl;
14         cout << ans << endl;
15         if (ans == '=') break;
16     }
17     return 0;
18 }

```

А программа `solve` например так:

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int l = -1, r = 1001;
5      while (l + 1 < r) {
6          int m = (l + r) / 2;
7          cerr << "Solve: ask=" << m << endl;
8          cout << m << endl;
9          char ans;
10         cin >> ans;
11         cerr << "Solve: got=" << ans << endl;
12         if (ans == '=') break;
13         else if (ans == '<') r = m;
14         else l = m;
15     }
16     return 0;
17 }

```

Чтобы всё это запустилось – нужно скомпилировать все три программы (пусть их имена `./run`, `./jury`, `./solve`) и запустить интерактор командой `./run`. А он уже сам запустит и программу жюри и программу-решение.

Здесь видно, что данные считываются и выводятся на стандартные потоки (`cin`, `cout`), но также делаются отладочные выводы в `cerr`. Отладочные выводы можно, при желании, делать в файл (использовать `ifstream`, `ofstream`) как и чтение каких-то дополнительных данных (в нашем случае можно было бы считывать из файла значение `secret`).

Не приятно, конечно, что метод не кроссплатформенный, но уж какой есть, так что стоит привыкать к Linux :)